Git 版控系統

Git Version Control System

November 7, 2026

WHUAI Team

Ostrichbeta Chan

1. 開始之前

2. 保存與檢查版本

3. branch 與合併

4. 分享與協作

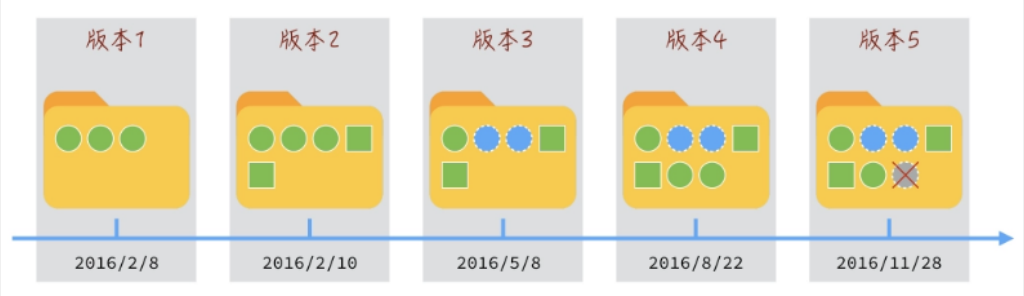
今天要完成的三件事

1. 保存、檢查並修正專案的版本。
2. 在 branch 上開發，並解決一次合併衝突。
3. 把 branch 推送到 GitHub，完成一次 Pull Request。

今天的方式

透過同一個小專案邊做邊學，每完成一段就動手練習。

我們想解決什麼問題？



- 哪個版本可以正常執行？
- 這次修改了什麼？為什麼？
- 兩個人的修改怎麼合在一起？

Git 保存版本；GitHub 協助分享與審閱

Git：電腦上的版本控制工具

- 保存 commit 與查看歷史。
- 建立 branch、合併修改。
- 本機操作不需要網路。

GitHub：託管 Git 倉庫的平臺

- 以遠端倉庫分享專案。
- 審閱程式碼、討論修改。
- 用 Pull Request 提出合併請求。

執行 commit 只會建立本機 commit；需 push 才會上傳。

先確認工具與 commit 作者資訊

```
git --version
git config --global user.name "Alex Chen"
git config --global user.email "alex@example.com"
```

- Git 的 user.name、user.email 是 commit 紀錄的作者資訊，不是 GitHub 登入資訊。
- Windows 預設沒有 Git；安裝 Git for Windows，使用 Git Bash。macOS、Linux 用終端機。
- 使用 Git 2.28 以上版本；建議事先確認可連線至 GitHub，並準備 GitHub 帳號。

指令約定

只輸入指令，不輸入提示符號；YOUR-USER 要換成自己的帳號。
可以向 AI 求助，但課堂上請親手輸入每一條命令。

1. 開始之前

2. 保存與檢查版本

3. branch 與合併

4. 分享與協作

今天只用這個小專案

```
def average(scores):
    return sum(scores) / len(scores)

print("Average:", average([70, 80, 90]))
```

- grades.py：計算平均分數。
- README.md：說明輸入與輸出。
- 把講義中的起始檔複製到**獨立的練習目錄**。

今天只用這個小專案

```
def average(scores):
    return sum(scores) / len(scores)

print("Average:", average([70, 80, 90]))
```

- `grades.py`：計算平均分數。
- `README.md`：說明輸入與輸出。
- 把講義中的起始檔複製到**獨立的練習目錄**。

重點不是範例程式怎麼運作，而是從檔案內容的變化學 Git。

一次 commit，經過三個地方



- 工作目錄：目前看到、正在修改的檔案。
- 暫存區：準備納入下一次 commit 的檔案內容。
- commit：把暫存區的專案狀態保存成一個版本。

讓練習目錄開始保存版本

```
pwd
git init -b main
git status
```

- 先確認目前在自己的 `git-grade-lab` 目錄。
- `git init -b main` 會在建立 repository 時把初始 branch 設為 `main`。
- 舊版 Git 或未指定初始 branch 的 `git init` 可能建立 `master`。
- **`.git` 保存 repository 的歷史與設定，請勿刪除。**

遇到問題，先讀 git status

讀懂三種常見狀態

`Untracked`	Git 還沒開始追蹤這個新檔案，尚未納入版本控制。
`Not staged`	已追蹤的檔案有修改，尚未加入暫存區。
`To be committed`	變更已進暫存區，下次 commit 會包含這些內容。

- 同一個檔案可以同時有已 staged 和尚未 staged 的變更。
- `working tree clean`：本機沒有待 commit 的變更，不代表已 push。

遇到問題，先讀 git status

下面是 `git status` 輸出的重點節錄，省略其他提示。

Untracked

```
Untracked files:
    README.md
    grades.py
```

Not staged

```
Changes not staged for commit:
    modified:   README.md
```

遇到問題，先讀 git status

To be committed

`git add README.md` 後，修改已暫存，等待 commit。

```
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   README.md
```

保存第一個版本

```
git add README.md grades.py
git status
git commit -m "Add grade calculator"
git status
```

- add：選擇這次要保存的內容。
- commit -m：保存版本，說明這次改了什麼。
- commit message 格式沒有強制規定，但內容要精確、簡潔。

先 add，再修改：哪一版會被 commit？

1. 在 README 加入 `Input: a list of scores.`
2. 執行 `git add README.md`。
3. 再加入 `Output: the average score.`，先不要 add。

```
git status
git diff
git diff --staged
```

- `diff`：工作目錄與暫存區的差異。
- `diff --staged`：暫存區與上次 commit 的差異。

練習 1：把兩次修改分開保存

1. 承接上一頁，只 commit Input 那一行。
2. 確認 Output 那一行還留在工作目錄。
3. 將 Output 那一行留到下一個 commit。

完成條件

- 至少三個 commit：初始專案、輸入說明、輸出說明。
- 工作目錄乾淨，並能說明兩次 diff 各自比較的對象。

有些檔案不應該進入版本紀錄

在專案根目錄建立 `.gitignore`：

```
.env
id_rsa
__pycache__/
build/
```

本機設定與憑證

- `.env`
- `id_rsa`

自動生成檔案

- `__pycache__/`
- `build/`

規則對尚未追蹤的檔案生效；已追蹤的檔案不會自動取消追蹤。

一起暫存目前目錄的變更

. 表示目前目錄；這個操作也包含子目錄。

```
git status
git add .
git diff --staged
```

- 新增、修改與刪除都會暫存；符合忽略規則的未追蹤檔案會略過。
- commit 前先讀 `git diff --staged`，確認要保存的變更。
- 只想選部分檔案時，仍可用 `git add 檔名`。

用歷史回答：改了什麼？為什麼？

```
git log --oneline
git show HEAD
```

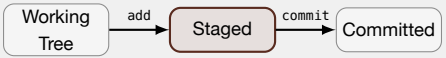
- `--oneline` 讓每個 commit 只佔一行，顯示縮短的 commit ID 與 message 首行。
- HEAD 通常經由目前 branch 指向該 branch 的最新 commit。
- `show HEAD` 可以檢查最近一次 commit 的內容。
- 出現分頁畫面時按 q 返回終端機。

修正錯誤前，先判斷問題在哪個階段

圖中亮色節點標出操作前相關變更的位置。

已 staged、未 commit：取消暫存並保留工作區檔案

```
git restore --staged README.md
```



Working Tree 未 staged 的修改：確定不要才丟棄

```
git restore README.md
```



已 commit 的變更：新增 revert commit 抵銷，保留歷史

```
git revert --no-edit HEAD
```



練習 2：撤銷錯誤 commit，保留歷史

1. 把 README 第一行改成 `# Broken calculator`。
2. 保存為一個 commit，訊息為 `Change heading by mistake`。
3. 先檢查歷史，再用 `git revert --no-edit HEAD` 抵銷這次 commit。

完成條件

標題恢復、工作目錄乾淨；歷史中同時保留錯誤 commit 與抵銷 commit。

revert 抵銷變更，並不刪除原始 commit。

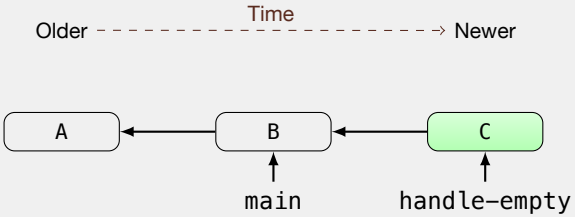
1. 開始之前

2. 保存與檢查版本

3. branch 與合併

4. 分享與協作

branch 代表另一條開發路線



- branch 名稱指向某個 commit；產生新 commit 時，目前 branch 會往前移動。
- 圖中功能 branch 多出 commit C，`main` 仍停在 B。
- HEAD 通常指向目前所在的 branch。

在功能 branch 處理空的分數清單

```
git switch -c handle-empty
```

把 average 函式改成：

```
def average(scores):
    if not scores:
        return 0
    return sum(scores) / len(scores)
```

```
git add grades.py
git commit -m "Handle empty score lists"
```

把功能合併回 main

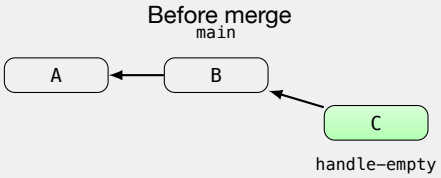
```
git switch main
git merge handle-empty
git log --oneline --graph --all
```

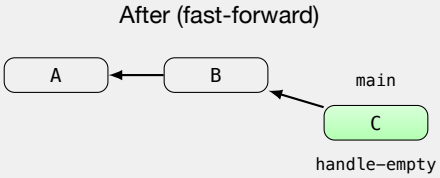
- `merge handle-empty` 把功能併入目前所在的 branch。
- 這次 `main` 沒有額外 commit，可以直接快轉（fast-forward）。
- 合併後功能 branch 名稱仍保留，不會自動刪除。

把功能合併回 main

```
git switch main
git merge handle-empty
git log --oneline --graph --all
```

- `merge handle-empty` 把功能併入目前所在的 branch。
- 這次 `main` 沒有額外 commit，可以直接快轉（fast-forward）。
- 合併後功能 branch 名稱仍保留，不會自動刪除。





練習 3：確認功能已合併回 main

1. 完成上一段的 branch 建立、修改、commit 與合併。
2. 用 `git branch` 確認目前在 `main`。
3. 開啟 `grades.py`，確認空清單的判斷已出現在 `main`。

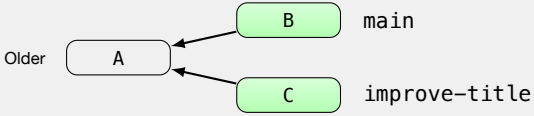
完成條件

`main` 已有新功能、工作目錄乾淨；能說明為何要先切換 branch 再合併。

可選檢查

有 Python 時，測試一般清單與空清單。

兩邊都修改同一個標題，怎麼辦？



`improve-title`

README 第一行改為：

```
# Team grade calculator
```

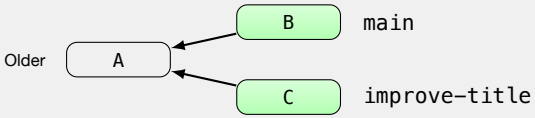
`main`

README 第一行改為：

```
# Class grade calculator
```

回到 `main`，執行 `git merge improve-title`。

兩邊都修改同一個標題，怎麼辦？



`improve-title`

README 第一行改為：

```
# Team grade calculator
```

`main`

README 第一行改為：

```
# Class grade calculator
```

回到 `main`，執行 `git merge improve-title`。

Git 無法判斷該保留哪個標題，需要我們自行決定。

讀懂衝突標記，保留雙方的用意

開啟發生衝突的 README.md，查看 Git 標出的衝突內容。

```
<<<<<<< HEAD
# Class grade calculator
=======
# Team grade calculator
>>>>>>> improve-title
```

- 上半部是目前 main 的內容，下半部是要併入的 branch 內容。
- 第一行改為 `# Team grade calculator for class`。
- 刪除全部衝突標記，保留其餘說明。

解決後，還要標記並完成 commit

```
git status
git add README.md
git commit -m "Combine team and class descriptions"
git status
```

- 編輯檔案並檢查結果。
- add 告訴 Git：這個檔案的衝突已處理。
- commit 完成合併；之後再檢查與驗證。
- 取消尚未完成的合併：`git merge --abort`。

練習 4：完成一次有衝突的合併

1. 依講義建立兩個修改標題的 commit，重現衝突。
2. 整合雙方意思，暫存並完成 merge commit。
3. 用 `git log --oneline --graph --all` 檢查歷史。

完成條件

- `main` 的標題保留 Team 與 class，其餘說明也要保留。
- 沒有衝突標記，工作目錄乾淨。
- 能解釋編輯後為何還要 add 與 commit。

1. 開始之前

2. 保存與檢查版本

3. branch 與合併

4. 分享與協作

遠端是另一份倉庫；origin 是它的別名

`main`	自己電腦上的 branch。
`origin`	本機給遠端網址取的別名。
`origin/main`	本機記錄的遠端 `main` 狀態，可能尚未更新。

- `push`：把本機 commit 送到遠端。
- `fetch`：取得遠端 commit，更新本機記錄的遠端狀態。
- fetch 不會直接更新目前工作目錄的檔案。

把自己的練習倉庫推到 GitHub

先在 GitHub 建立 `git-grade-lab` 空倉庫，先不新增 README、授權或.gitignore。



- 本機已有 repository，請看 GitHub 空倉庫頁的 push an existing repository 區塊。
- 圖片是範例，請複製自己 repository 的 URL，不要照抄圖片中的帳號。

```
git remote add origin https://github.com/YOUR-USER/git-grade-lab.git
git push -u origin main
```

課堂已使用 `main`，GitHub 指示的 `git branch -M main` 不必重做。

取得既有專案，使用 clone

```
git clone https://github.com/YOUR-USER/git-grade-lab.git git-grade-copy
cd git-grade-copy
git status
```

- clone 會取得倉庫，並設定 origin 與預設 branch 的上游。
- 複製好之後不需要再 init。
- 回到**原本練習目錄**繼續下一段。

只有遠端有新增 commit、自己沒有額外 commit 時，可以：

```
git pull --ff-only
```

push 被拒絕：先整合遠端更新

情境：在 `main`，兩邊各有對方沒有的 commit，工作目錄乾淨。

```
git fetch origin
git merge --no-edit origin/main
git push
```

- 沒有內容衝突時，Git 可以直接整合。
- 若有衝突，依序編輯、add、commit，再 push。
- `--no-edit` 接受自動產生的合併訊息。

兩邊各有新 commit，不一定代表內容衝突。

用 Pull Request 讓同學審閱修改

```
git switch -c docs-usage
```

在 README 加上使用說明後：

```
git add README.md
git commit -m "Explain how to run the calculator"
git push -u origin docs-usage
```

- 在 GitHub 開 PR：base 選 `main`，compare 選 `docs-usage`。
- 說明目的、內容與檢查方式，請同學審閱。
- 修改後繼續 commit 並 push 到同一 branch，原本的 PR 就會更新。

練習 5：兩人完成一次審閱與合併

1. 各自在自己的倉庫提出 `docs-usage` 的 PR。
2. 交換連結，檢查用法是否清楚，留下具體建議。
3. 作者回應並更新同一個 PR，再由有權限的人合併。
4. 回到本機 `main`，取得合併後的版本。

```
git switch main
git pull --ff-only
```

完成條件

PR 有說明與審閱紀錄；本機 `main` 有使用說明，工作目錄乾淨。

課後想一想

1. add 後又改檔案，下次 commit 會保存哪一版？
2. commit 成功後，GitHub 就一定看得到嗎？
3. 把 feature 合進 `main`，應該先切到哪個 branch？
4. 編輯完衝突檔案，還需要哪兩個步驟？
5. PR 收到意見後，要怎麼更新同一個 PR？

日常工作：先看狀態，再選下一步

保存與檢查

- `git status`
- `git diff`
- `git diff --staged`
- `git add 檔名`
- `git commit -m " 說明"`
- `git log --oneline`

branch 與協作

- `git switch -c branch 名稱`
- `git switch main`
- `git merge branch 名稱`
- `git push -u origin branch 名稱`
- `git pull --ff-only`

5. 附錄

課前準備：工具、編輯器與認證

- 安裝 Git，選好編輯器，確認自己知道如何保存檔案。
- 若使用 VS Code 且已安裝 `code` 命令，可設定：

```
git config --global core.editor "code --wait"
```

- HTTPS 認證：Git Credential Manager 或 GitHub CLI。
- 用個人的測試倉庫確認可以 push；詳細流程見講義。
- 若已有其他可用的編輯器與認證方式，可以繼續使用。

Token 不要放在程式碼、遠端網址或截圖中。

先把基本流程練熟，再學進階操作

- `stash`：暫時收起尚未 commit 的工作。
- `rebase / cherry-pick`：重整或搬移 commit。
- `reset / reflog`：移動 branch、查找本機操作紀錄。

恢復能力有邊界

- Git 沒有保存過的內容，不能保證找回。
- hard reset 會覆蓋工作目錄內容，本次練習不需要用到它。
- 改寫已分享的歷史前，先與團隊確認工作方式。

課後練習與官方參考

- Pro Git：繁體中文版本
- Git diff：比較未暫存與已暫存的內容
- Git restore、Git revert、gitignore
- GitHub：設定 Git 認證
- GitHub：Pull Request 與程式碼審閱

下次拿自己的小專案再做一次

建立 branch，完成一項小修改，檢查 diff，commit 並請同學審閱。